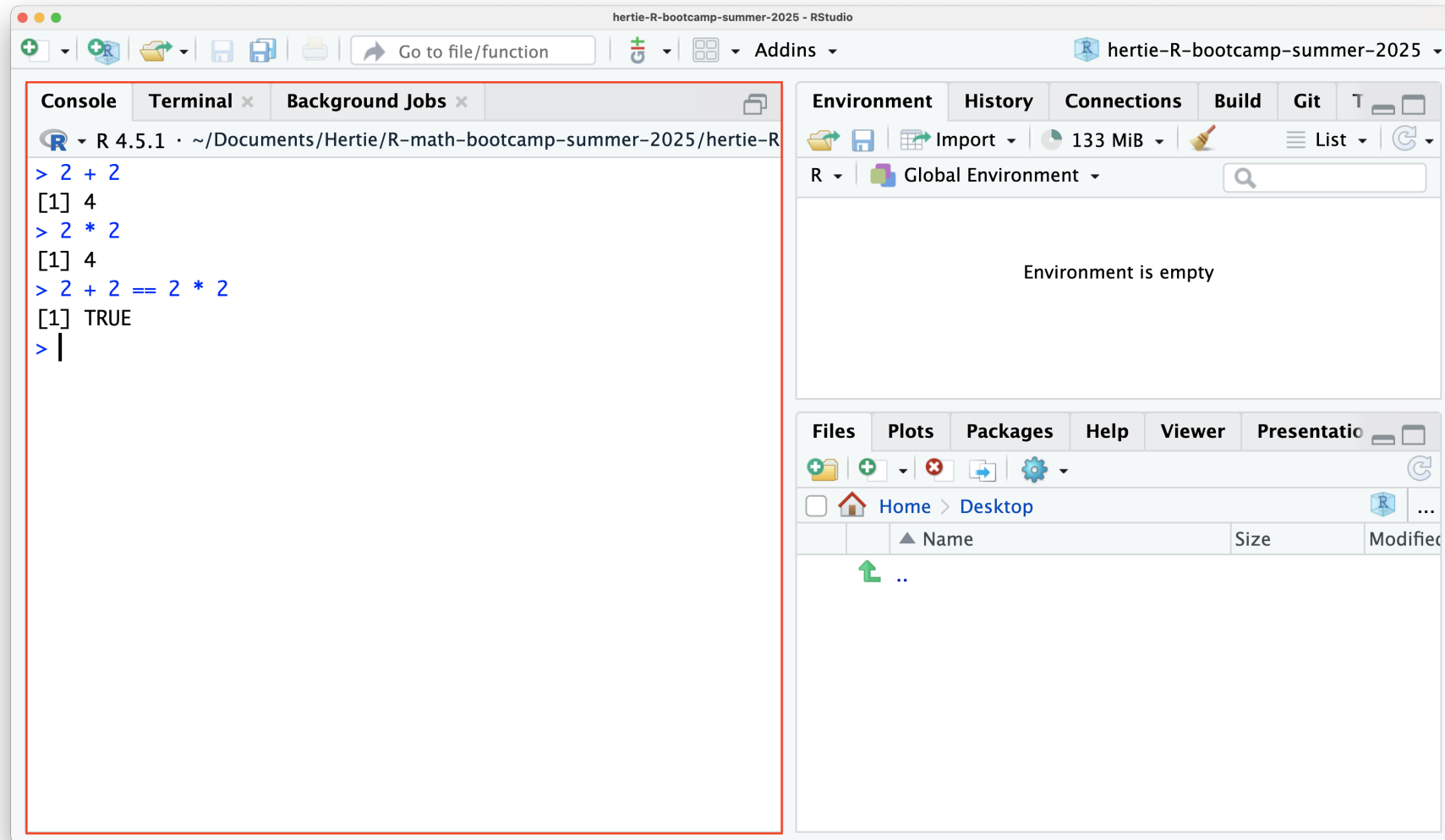


Introduction to R and RStudio

Linus Hagemann

Using R as a Calculator

The Console in RStudio



The red box is the console (also known as REPL) where you can run R code interactively.

R Operators

R includes many operators, including

Arithmetic

- `+` : Addition
- `-` : Subtraction
- `*` : Multiplication
- `/` : Division
- `^` : Exponentiation

Relational

- `==` : Equal to
- `!=` : Not equal to
- `<` : Less than
- `>` : Greater than
- `<=` : Less than or equal to
- `>=` : Greater than or equal to

Logical

- `&` : AND
- `|` : OR
- `!` : NOT

Exercises

1. Calculate the number of hours in a year.
2. Calculate the population density of Berlin (Let's say 3897145 persons in 891.3 km^2)

Solution

```
1 365 * 24
```

```
[1] 8760
```

```
1 3897145 / 891.3
```

```
[1] 4372.428
```

Variables (/Objects)

Variables are values assigned to names using the assignment operator `<-`. For example

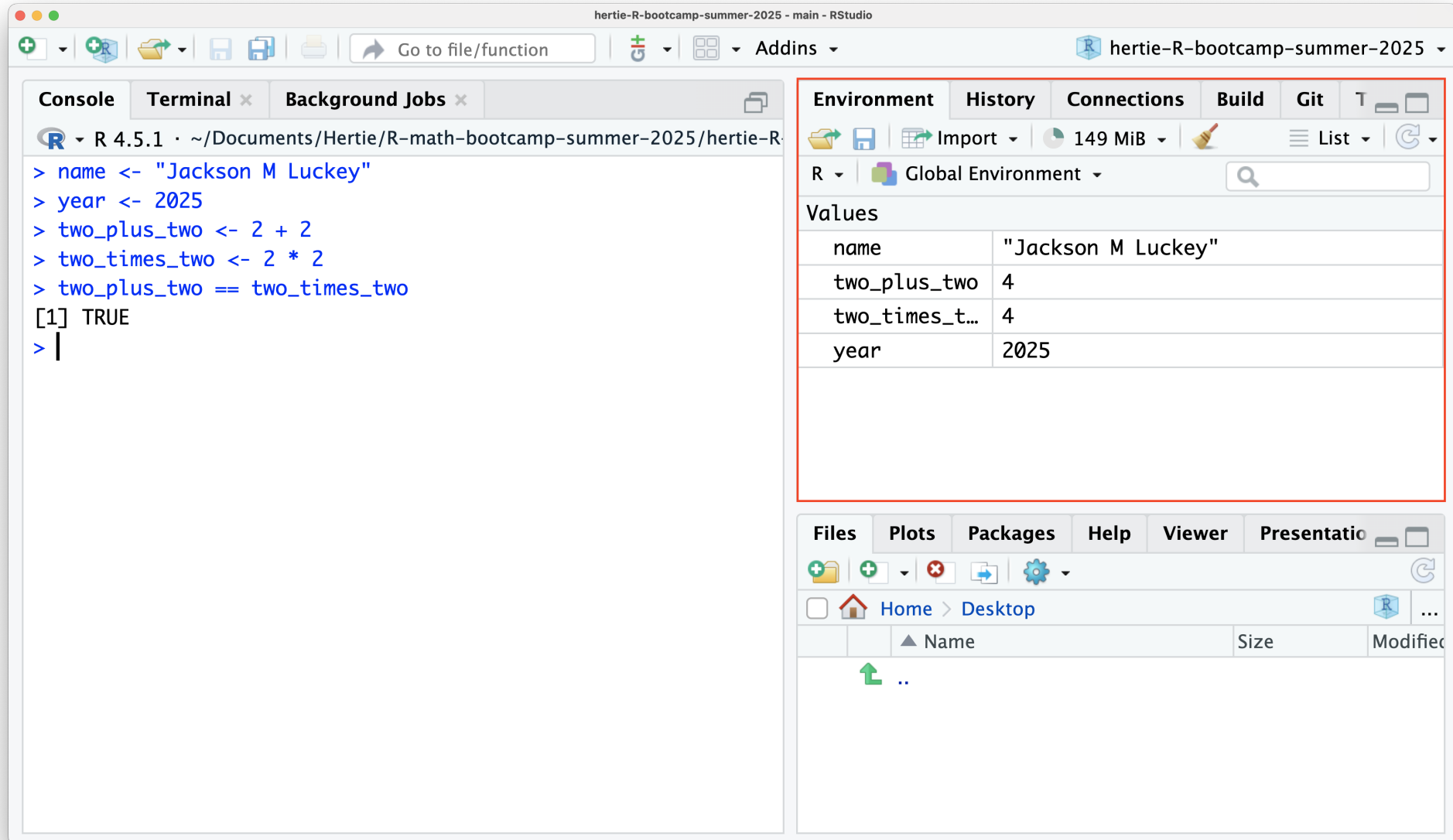
```
1 name <- "Introduction to Data Science"
2 year <- 2025
3 two_plus_two <- 2 + 2
4 two_times_two <- 2 * 2
```

We can then use these objects:

```
1 two_plus_two == two_times_two
```

```
[1] TRUE
```

The Environment Pane in RStudio



The screenshot displays the RStudio interface with the following components:

- Console:** Shows the execution of R code:

```
> name <- "Jackson M Luckey"
> year <- 2025
> two_plus_two <- 2 + 2
> two_times_two <- 2 * 2
> two_plus_two == two_times_two
[1] TRUE
> |
```
- Environment Pane:** Located on the right, it shows the current environment (Global Environment) with the following variables and values:

Variable	Value
name	"Jackson M Luckey"
two_plus_two	4
two_times_two	4
year	2025
- Files Pane:** Located at the bottom right, it shows the file explorer with the current directory set to Home > Desktop. The file list includes a green upward arrow and the text "..".

Exercise

Using objects, calculate the population density of Berlin (3897145 persons in 891.3 km^2) along the lines of

```
1 days_in_year <- 365
2 hours_in_day <- 24
3 hours_in_year <- days_in_year * hours_in_day
4
5 hours_in_year
```

```
[1] 8760
```

Solution

```
1 berlin_pop <- 3897145
2 berlin_area <- 891.3
3 berlin_pop_density <- berlin_pop / berlin_area
4
5 berlin_pop_density
```

```
[1] 4372.428
```

What kinds of objects are there?

Objects can represent different things, like text (“character”), numbers (“numeric”), and functions (“function”). Throughout the bootcamp and the upcoming semester you’ll learn about many other kinds of objects.

```
1 class("Hello")
```

```
[1] "character"
```

```
1 class(berlin_pop_density)
```

```
[1] "numeric"
```

```
1 class(sqrt)
```

```
[1] "function"
```

Vectors

Vectors are objects that represent an ordered collection of information. You can create a vector using `c()` as shown below

```
1 years <- c(2020, 2021, 2022, 2023, 2024, 2025)
2 cities <- c("Berlin", "Paris", "London", "Beijing", "Tokyo")
```

Vectors

You can also combine vectors using `c()` like

```
1 north_american_cities <- c("Mexico City", "New York City", "San Francisco",  
2 south_american_cities <- c("Rio de Janeiro", "Lima", "Buenos Aires", "Medel  
3 american_cities <- c(north_american_cities, south_american_cities)  
4 american_cities
```

```
[1] "Mexico City"      "New York City"    "San Francisco"    "Vancouver"  
[5] "Rio de Janeiro"  "Lima"             "Buenos Aires"     "Medellin"
```

Vectors are always one dimensional.

Vectors

You can manipulate vectors just like individual numbers. For example, we can get the number of seconds since 1970 (a common way of storing dates/time in computers) with

```
1 seconds_in_hour <- 60 * 60
2
3 seconds_since_1970_vec <- (years - 1970) * hours_in_year * seconds_in_hour
```

Exercise

Calculate the population density of Berlin for every year represented in `berlin_pop`.

```
1 # Population of Berlin in 1980, 1990, 2000, 2010, 2020
2 years      <- c(1980, 1990, 2000, 2010, 2020)
3 berlin_pop <- c(3048759, 3433695, 3382169, 3460725, 3664088)
```


Solution

```
1 berlin_pop / berlin_area
```

```
[1] 3420.576 3852.457 3794.647 3882.784 4110.948
```

Indexing Vectors

You can access elements inside a vector using `[]`. For example, to get the first element in `cities` (“Berlin”), do

```
1 cities[1]
```

```
[1] "Berlin"
```

In R, counting and indexing start from 1, unlike many other programming languages like Python.

Exercise

Get the population of Berlin in 2020 from `berlin_pop`.

Solution

```
1 berlin_pop[5]
```

```
[1] 3664088
```

Functions

Functions are another kind of object in R. Many functions take objects as arguments and return an object. For example, `c()` is a function.

Other simple and common functions that work on vectors include:

- `length()`: Returns the number of elements inside an object.
- `min()`: Returns the smallest value in an object.
- `max()`: Returns the largest value in an object.
- `sum()`: Returns the sum of all elements in an object.
- `mean()`: Returns the average (mean) of all elements in an

Functions

For example, we can find the average population of Berlin between 1980 and 2020 by either doing

```
1 mean(berlin_pop)
```

```
[1] 3397887
```

or

```
1 sum(berlin_pop) / length(berlin_pop)
```

```
[1] 3397887
```

Exercise

Using `min()` and `max()`, find the lowest and highest population density in Berlin between 1980 and 2020.

Solution

```
1 min(berlin_pop) / berlin_area
```

```
[1] 3420.576
```

```
1 max(berlin_pop) / berlin_area
```

```
[1] 4110.948
```


Dataframes

Dataframes

Dataframes are objects that have columns (conventionally variables) and rows (conventionally observations). Think of a spreadsheet like Excel or Google Sheets.

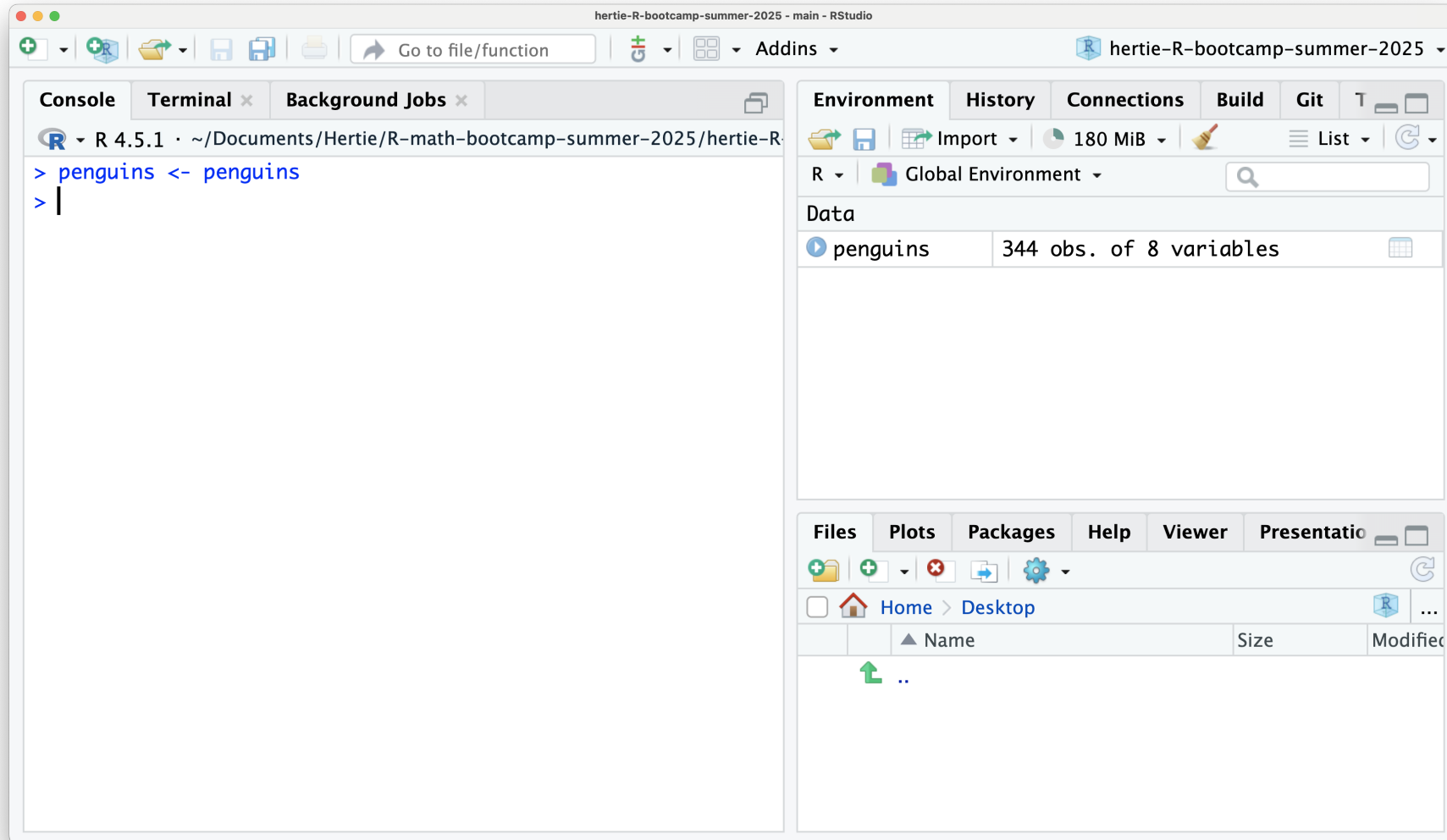
R has several dataframes, including `penguins`, built in. We can load penguins into R with

```
1 penguins <- penguins
```

and view the dataframe with

```
1 View(penguins)
```

Loading Penguins



`penguins <- penguins` loads the penguin dataset into R, so it appears in the environment pane.

Viewing Penguins

The screenshot shows the RStudio interface with the 'penguins' dataset loaded. The main pane displays a data frame with 10 rows and 8 columns. The columns are: species, island, bill_len, bill_dep, flipper_len, body_mass, sex, and year. The data shows Adelie penguins from Torgersen island, with various measurements and years (2007). The console shows the commands used to load and view the dataset.

	species	island	bill_len	bill_dep	flipper_len	body_mass	sex	year
1	Adelie	Torgersen	39.1	18.7	181	3750	male	2007
2	Adelie	Torgersen	39.5	17.4	186	3800	female	2007
3	Adelie	Torgersen	40.3	18.0	195	3250	female	2007
4	Adelie	Torgersen	NA	NA	NA	NA	NA	2007
5	Adelie	Torgersen	36.7	19.3	193	3450	female	2007
6	Adelie	Torgersen	39.3	20.6	190	3650	male	2007
7	Adelie	Torgersen	38.9	17.8	181	3625	female	2007
8	Adelie	Torgersen	39.2	19.6	195	4675	male	2007
9	Adelie	Torgersen	34.1	18.1	193	3475	NA	2007
10	Adelie	Torgersen	42.0	20.2	190	4250	NA	2007

Showing 1 to 10 of 344 entries, 8 total columns

```
R > R 4.5.1 · ~/Documents/Hertie/R-math-bootcamp-summer-2025/hertie-R-bootcamp-summer-2025/  
> penguins <- penguins  
> View(penguins)  
> |
```

`View(penguins)` opens a new pane that shows the penguins dataset.

Dataframe Columns as Vectors

You can access dataframe columns as vectors using `$`.

For example, we can get `body_mass` from `penguins` with

```
1 penguins$body_mass
[1] 3750 3800 3250 NA 3450 3650 3625 4675 3475 4250 3300 3700 3200 3800
4400
[16] 3700 3450 4500 3325 4200 3400 3600 3800 3950 3800 3800 3550 3200 3150
3950
[31] 3250 3900 3300 3900 3325 4150 3950 3550 3300 4650 3150 3900 3100 4400
3000
[46] 4600 3425 2975 3450 4150 3500 4300 3450 4050 2900 3700 3550 3800 2850
3750
[61] 3150 4400 3600 4050 2850 3950 3350 4100 3050 4450 3600 3900 3550 4150
3700
[76] 4250 3700 3900 3550 4000 3200 4700 3800 4200 3350 3550 3800 3500 3950
3600
[91] 3550 4300 3400 4450 3300 4300 3700 4350 2900 4100 3725 4725 3075 4250
2925
- -
```

Exercise

What is the minimum, mean, and maximum body mass of a penguin in the `penguins` dataset?

```
1 min(penguins$body_mass, na.rm = TRUE)
```

```
[1] 2700
```

```
1 mean(penguins$body_mass)
```

```
[1] NA
```

```
1 max(penguins$body_mass)
```

```
[1] NA
```